NLOS Detection Signal Processing 공유

Note

[Stationary, Moving Vehicle Scenario] NLOS Detection

- Radar는 Occluder로 인해 보이지 않는 물체를 NLOS (multipath) 신호를 통해 알 수 있음
- Point Cloud를 봤을 때 주변 환경 인식 가능
- Doppler를 통해 주변 환경이나 움직이는 사람 등을 구분할 수 있음
- Doppler 가 없는 Static한 물체와 Doppler를 가지는 dynamic한 물체를 구분하여 처리

0. Import Libraries

- REMILAB_CFAR, REMILAB_DoA, REMILAB_coordinates2 라이브러리는 파일이 같은 디렉토리 안에 있어야 실행이 가능

```
import numpy as np
import matplotlib.pyplot as plt
from matplotlib import gridspec
from matplotlib.projections import register_projection
from matplotlib import cm
from matplotlib.ticker import LinearLocator
import matplotlib.image as img

import warnings
import operator
import math
import os
import imageio

import datetime
import shutil
import math

import bagpy
from bagpy import bagreader
import pandas as pd

from REMILAB_CFAR_v2 import *
from REMILAB_DoA import *
from REMILAB_coordinates2 import *

import copy
import csv
from PIL import Image
import re

warnings.filterwarnings('ignore')
```

```
np.seterr(divide = 'ignore')
from ipywidgets import interact, jslink
```

1. Load Files

- zip file 풀어서 이름 그대로 폴더 형태로 코드랑 같은 폴더에 넣어서 사용
- File 이름에 맞춰서 변경해서 사용
  - radar_no = 0: MIMO Radar (Tx3, Rx4)
  - radar_no = 1: Beamforming Radar (Tx1, Rx4)
  - df_radar: Pandas의 dataframe 구조
    - Rows: Time, data_0, data_1, ..., data_n

```
# ex) Site1_Scene1_Stop_Case1_Trial5_9m
# experiment_date = '0104'
experiment_position = 'Site1'
scene_folder_name = 'Scene1'
vehicle_status = 'Stop'
case_name = 'Case1'
filelabel='Trial3_9m'
radar_no = 'radar_0'

folder_name=f'{experiment_position}_{scene_folder_name}_{vehicle_status}'
data_folder_name=f'{folder_name}_{case_name}_{filelabel}'
data_file_name=f'{folder_name}_{case_name}_{filelabel}_{radar_no}'

#%%

load_csv_path='radar30-radar_data'
try:
    # b =
bagreader(f'./{folder_name}/{data_folder_name}/{data_folder_name}/'+data_file_name+'.bag')
    b = bagreader(f'./{data_folder_name}/{data_file_name}.bag')

    print(b.topic_table)
    csvfiles = []

    for t in b.topics:
        data = b.message_by_topic(t)
        csvfiles.append(data)

    df_radar = pd.read_csv(f'./{data_folder_name}/{data_file_name}/{load_csv_path}.csv')

    # df_radar =
pd.read_csv(f'./{folder_name}/{data_folder_name}/{data_folder_name}/'+data_file_name+'/'+loa
d_csv_path+'.csv')

except FileNotFoundError:
    load_csv_path = 'radar31-radar_data_1'
    df_radar = pd.read_csv(f'./{data_folder_name}/{data_file_name}/{load_csv_path}.csv')
    # df_radar =
pd.read_csv(f'./{folder_name}/{data_folder_name}/{data_folder_name}/'+data_file_name+'/'+loa
d_csv_path+'.csv')
```

```
print(type(df_radar))
```

2. Setting Radar Cube Parameter

- Radar cube parameter 설정
  - numSample: Chirp 1개의 sample 개수
  - numChirp: Frame 1개의 chirp 개수
  - numRx: 레이다의 Rx 개수
  - numTx: 레이다의 Tx 개수
    - MIMO Radar (radar_no=0) = 3개
    - Beamforming Radar (radar_n0=1) = 1개
  - numDirection: Antenna virtual array 개수 (numTx * numRx)
  - numTimestamp: Timestamp 개수
  - numFrame: Frame 개수
  - numRealSample: AWR2944는 Real receiver, Sample 개수의 1/2배

```
numSample = 512
numChirp = 64
numRx = 4
if radar_no == 'radar_0':
    numTx = 3
elif radar_no == 'radar_1':
    numTx = 1
numDirection = numRx * numTx
numTimestamp = df_radar.shape[0]
numFrame = df_radar.shape[0]
numRealSample = int(numSample/2) # Real Signal
```

3. Data Pre-processing

- chirpIDX: Chirp index
  - in Static Scene: Chirp idx = 32 (Doppler = 0) --> 정지한 물체 (ex 벽 같은 LOS Clutter)
  - in Dynamic Scene: Chirp idx = 29 --> 움직이는 물체 (NLOS Target)
- frameIDX: Frame index = 50
- timestamp: Point Cloud를 위해 tim stamp 저장
  - Radar Cube 만들 때는 timestamp 제거 후 reconstruction 해야 함 (drop_timestamp)

```
chirpIDX = 32
frameIDX = 50

save_timestamp = []
save_timestamp = df_radar.iloc[frameIDX-1, 0]
drop_timestamp = df_radar.drop('Time', axis='columns')
```

4. Radar Cube Reconstruction

- Raveling (np.ravel): 다차원 배열을 1차원으로 푸는 것
  - order = 'F': Column-major ordering
- Radar Cube
  - Shape: 5차원 cube (Frame, Sample, Rx, Tx, Chirp)
  - Raveled_data를 Frame -> Sample -> Rx -> Tx -> Chirp 순으로 Reshaping
- Radar_cube_azimuth: Rx, Tx 합쳐서 Virtual array cube
  - Shape: 4차원 cube (Frame, Sample, Direction, Chirp)
    - (103, 512, 12, 64)
  - Tx -> Rx 순으로 Virtual antenna array 생성
    - Tx0 Rx0 -> Tx0 Rx1 -> Tx0 Rx2 -> ... -> Tx2 Rx0 -> Tx2 Rx1 -> Tx2 Rx2 -> Tx2 Rx3

```
# Raveling
raveled_data = np.ravel(drop_timestamp, order='F')
raveled_data.shape

radar_cube = np.reshape(raveled_data, (numFrame, numSample, numRx, numTx, numChirp),
                                                    order='F')
radar_cube_azimuth = np.zeros((numFrame, numSample, numDirection, numChirp))

# Radar Cube Reshaping
count = 0

for j in range(numTx): # Tx
    for k in range(numRx): # Rx
        radar_cube_azimuth[:, :, count, :] = radar_cube[:, :, k, j, :]
        count = count + 1
```

5. Signal Processing Function Definition

- Range -> Doppler -> Angle FFT 순으로 신호 처리 진행

1. Range FFT
   - Radar의 Rx, Tx Mixing -> 주파수 차이 Beat Frequency -> FFT -> Range
   - Range Estimation을 하기 위해 Sample 방향으로 FFT Processing
   - np.fft.rfft (Real signal FFT): Real 신호는 fft 데이터가 양, 음의 주파수로 똑같이 나오므로 반만 사용
     - axis = 1 (Sample) 방향
     - Real 신호를 처리하여 주파수를 반만 썼기 때문에 fftshift는 필요 없음

```
def range_FFT(input_data, numframe, numsample, numdirection, numchirp):
    output_data = np.zeros((numframe, int(numsample/2), numdirection, numchirp),
    dtype ='complex')
    output_data[:, :, :, :] = np.fft.rfft(input_data[:, :, :, :], axis = 1)[:,
:numRealSample, :, :]
    return output_data
```

2. Doppler FFT
   - Range가 거의 같을 때는 Rx chirp마다 작은 위상 변화가 생김
   - 두 IF 신호의 위상차 -> 물체의 Velocity Estimation

- Doppler Estimation을 하기 위해 Chirp 방향으로 FFT Processing
  - Radar와 같은 방향으로 움직이는 경우 - Positive Doppler Frequency
  - Radar와 반대 방향으로 움직이는 경우 - Negative Doppler Frequency
- np.fft.fft: Complex data 처리
  - axis = 3 (Chirp) 방향
  - Complex 신호를 처리했기 때문에 fftshift 필요

```
def doppler_FFT(input_data, numframe, numsample, numdirection, numchirp):
    output_data = np.zeros((numframe, numsample, numdirection, numchirp),
    dtype ='complex')
    output_data[:, :, :, :] = np.fft.fft(input_data[:, :, :, :numChirp], axis = 3)
    return output_data
```

3. Direction FFT
   - Velocity가 거의 같은 때는 실제 안테나 간의 물리적인 거리차로 인해 위상 변화가 생김
   - 공간적 위상차 -> 물체의 Angle Estimation
   - Angle Estimation을 하기 위해 Antenna Virtual Array 방향으로 FFT Processing
     - Radar 기준 0도 ~ 90도 사이의 경우 - Positive
     - Radar 기준 -90도 ~0도 사이의 경우 - Negative
   - **해상도 때문에 Direction FFT 보단 High Resolution DoA Processing을 많이 씀**
   - np.fft.fft: Complex data 처리
     - axis = 2 (Antenna Virtual Array) 방향
     - Complex 신호를 처리 했기 때문에 fftshift 필요

```
def direction_FFT(input_data, numframe, numsample, numdirection, numchirp):
    output_data = np.zeros((numframe, numsample, numdirection, numchirp),
    dtype ='complex')
    output_data[:, :, :, :] = np.fft.fft(input_data[:, :, :, :], axis = 2)
    return output_data
```

4. FFT Shift
   - Complex data를 FFT 했을 때 0Hz를 중심으로 대칭이 되도록 주파수 재배열
   - Fourier Transform Result -> FFT Shift

```
def FFT_shift(input_data, numframe, numsample, numdirection, numchirp):

    output_data_shift = np.zeros((numframe, numsample, numdirection, numchirp),
    dtype='complex')
    output_data_shift = np.fft.fftshift(input_data, 3) # Chirp 방향

    return output_data_shift
```

5. Dynamic Scene에서만 사용하는 Processing
   - MTI(Moving Target Indicator) Filter
     - Dynamic Object와 Static Object를 구별
     - Doppler 영역에서 뒤에서 앞의 성분을 빼서 **Static Clutter 제거 -> Moving target만 남김**
       - Static한 물체: 클러터의 echo 신호 사이에 위상차가 없음 Doppler=0

        - Dynamic한 물체: 도플러의 영향으로 echo 신호 사이에 위상차 발생 Doppler != 0
    - 신호에서 움직임이 있는 성분 (노이즈 및 이동 표적) 제거시에는 IMTI Filter 사용

```
def mti_filter(input_data):
    filtered_data = np.zeros_like(input_data)
    for i in range(1, input_data.shape[3]):
        filtered_data[:,:,:,i] = input_data[:,:,:,i] - input_data[:,:,:,i-1]
    return filtered_data
```

- - Add All Directions (AAD)
        - Target을 검출하기 위해 Antenna 방향으로 다 더해서 신호의 크기를 크게함
            - 안테나끼리는 위상차가 작음 -> Direction 하나로
        - 모든 방향에 대하여 CFAR를 적용하기 위함
        - Absolute value로 처리한 후 AAD 처리

```
def add_all_directions(input_data, numframe, numsample, numchirp):
    after_adding_all_directions = np.zeros((numframe, numsample, 1, numchirp),
    dtype = 'complex')
    after_adding_all_directions = np.sum(input_data, axis=2, keepdims=True)
    return after_adding_all_directions
```

Info

Proposed Detection Algorithm
!! Frame 마다 적용 !!

1. Static Scene

- Range FFT -> Doppler FFT -> Chirp index of zero velocity -> Bartlett (DoA Proecssing) -> 2D OS-CFAR -> Static Scene Point Cloud

2. Dynamic Scene

- Range FFT -> MTI Method -> Doppler FFT -> Absolute value -> Add all directions -> OF CFAR in Doppler (2D CFAR) -> Chirp index of Moving Object (연산이 너무 많아 Moving Object가 존재하는 Chirp만 뽑아서 Bartlett 연산)
- Range FFT -> MTI Method -> Doppler FFT -> Moving Object Chirp index -> Bartlett (DoA Processing) -> 1D OS-CFAR -> Dynamic Scene Point Cloud

6. Bartlett Processing Function Definition

- Moving Target
    - Bartlett 알고리즘 사용
        - Barltett Processing
            - virtual_array = Antenna virtual array 개수
            - lamb = Antenna distance, 0.5lambda (Uniform Linear Array)

        - numAngle = Beam 개수
    - 안테나로 수신한 신호들을 Steering Vector와 내적하여 각도 성분 추출
        - Bartlett Processing
- CFAR Detection으로 얻은 Moving target chirp index (range point와 doppler point)에 Bartlett processing 적용

```
def apply_bartlett_moving_target(types_of_scenes, CFAR_range_point, CFAR_doppler_point,
frame_index):

    global bartlett_reshape_cube
    global DoA
    global incident_angles

    virtual_array = numDirection
    numAngle = 181

    bartlett_reshape_cube = np.zeros((len(CFAR_range_point), numAngle), dtype='complex')

    for i in range(len(CFAR_range_point)):

        range_index = CFAR_range_point[i]
        chirp_idx = CFAR_doppler_point[i]

                # Correlation Matrix
        radar_co_matrix = np.zeros((1, virtual_array), dtype='complex')

        for j in range(virtual_array):
            radar_co_matrix[:, j] = types_of_scenes[frame_index, range_index, j,
            chirp_idx]

        radar_co_matrix = np.transpose(radar_co_matrix, (1, 0))
        radar_correlation_cube = corr_matrix_estimate(radar_co_matrix.T, imp='fast')

        lamb = 0.5
        array_alignment = np.arange(0, virtual_array, 1) * lamb
        incident_angles = np.arange(0, 181, 1)

                # Steering Vector
        scanning_vectors = gen_ula_scanning_vectors(array_alignment, incident_angles)
        # 얻은 각도 성분
        DoA = DOA_Bartlett(radar_correlation_cube, scanning_vectors)

        bartlett_reshape_cube[i, :] = DoA

    return bartlett_reshape_cube, DoA, incident_angles
```

- Static Clutter
    - Bartlett 알고리즘 사용
        - Zero velocity를 가지는 Chirp index를 갖고 Bartlett Processing

```
def apply_bartlett_static(types_of_scenes, chirp_index, frame_index):

    global bartlett_reshape_cube
```

```
    global DoA
    global incident_angles

    virtual_array = numDirection
    numAngle = 181

    bartlett_reshape_cube = np.zeros((numRealSample, numAngle), dtype = 'complex')

    for range_index in range(numRealSample):
        radar_co_matrix = np.zeros((1, virtual_array), dtype = 'complex')

        for i in range(virtual_array):
            radar_co_matrix[:, i] = types_of_scenes[frame_index, range_index, i,
            chirp_index]

        radar_co_matrix = np.transpose(radar_co_matrix, (1, 0))
        radar_correlation_cube = corr_matrix_estimate(radar_co_matrix.T, imp='fast')

        lamb = 0.5
        array_alignment = np.arange(0, virtual_array, 1) * lamb
        incident_angles = np.arange(0, 181, 1)

        scanning_vectors = gen_ula_scanning_vectors(array_alignment, incident_angles)
        DoA = DOA_Bartlett(radar_correlation_cube, scanning_vectors)
        bartlett_reshape_cube[range_index, :] = DoA

    return bartlett_reshape_cube, DoA, incident_angles
```

7. Coordinate Transformation

- Map을 만들기 위해서 센서의 절대적인 위치 필요
  - Local Coordinate: 센서를 중심으로 그려진 Coordinate
  - Global Coordinate: 센서의 절대적인 위치를 기준으로 그려진 Coordinate
    - Point가 아닌 **Axis를 회전**
      - Axis 반시계 회전 = Point theta 시계 회전 = Point -theta 반시계 회전

$$T_{l2g,axis} = \begin{pmatrix} cosm_\phi & sinm_\phi & 0 \\ -sinm_\phi & cosm_\phi & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

    - Shearing Matrix
      - 기준점 옮길 때 사용 (ex 차량 기준 레이다 마운팅 위치때문에 사용)
      - Point 기준 계산 -> 기존 point (p_x, p_y)를 (t_x, t_y)만큼 이동

$$T_{l2g,moving} = \begin{pmatrix} 1 & 0 & m_x \\ 0 & 1 & m_y \\ 0 & 0 & 1 \end{pmatrix}$$

  - Transformation Matrix
    - 실험에 사용한 Palisade 차종을 기준으로 파라미터 선정
      $$T{l2g}=\begin{equation}
      \begin{pmatrix}
      cos{m\phi} & sin{m\phi} & m_x \\
      -sin{m\phi} & cos{m_\phi} & m_y \\

0 & 0 & 1 \
\end{pmatrix}
\end{equation}

$-\$m_x\$ = width, \$m_y\$ = wheelbase, \$m_\phi\$ = theta//Radar는정면코너에mount되어있음$

- Parameter for Coordinate Transformation
  - overall*width = 2*m_x , wheelbase = m_y , theta_rad = $m*ϕ$ [rad]

```
overall_width = 1.975
wheelbase = 3.77 # normal = 2.9 / 3.77
theta_rad= 30 * (np.pi/180) # (-30-90)*(np.pi/180) # need to change
yaw_rate = 0
```

- Coordinate Transformation
  - Radar Mounting 위치에 맞춰 Point Cloud Transformation

```
def global_coordinate(before_cal, overall_width, wheelbase, theta_rad, yaw_rate):

    '''
    Function for transformate local to global coordinate

    before_cal : local coordinate(x, y)
    theta_rad : mounted angle of sensor
                    mounted left side -
                    mounted right side +

    '''

    x_y_direction_nocal = np.zeros((len(before_cal), 3), dtype=float)

    x_y_direction_nocal[:,:2]=before_cal.iloc[:,:2].to_numpy() #cosine theta array

    x_y_direction_nocal[:,2]=1

    point_sc = x_y_direction_nocal.T
    trans_mat = np.zeros((3,3), dtype=float)
    trans_mat = ([math.cos(theta_rad), math.sin(theta_rad), (overall_width/2)],
                 [-math.sin(theta_rad), math.cos(theta_rad), wheelbase],
                 [0, 0, 1]) # rotate axis
    point_cc = trans_mat@point_sc

    return point_cc
```

8. 2D OS CFAR in Doppler (Dynamic Scene)

- CFAR (Constant False Alarm Rate) Algorithm
  - 1D/2D Radar image에서 특정 위치에서의 표적 유무를 판단하는 기법
    - CA-CFAR (평균), OS-CFAR (정렬)
  - Target에 대하여 False alarm을 하지 않도록 thresold를 고정시키지 않고 window를 나눠 유동적으로 하여 false alarm을 하지 않게 사용하는 알고리즘
  - CFAR 성능

        - Guard cell size와 표적의 size가 유사해야 좋은 성능을 냄
        - 모든 신호에 적용 가능한 최적의 reference cell size가 있다고 보기 어려움
        - 다루는 신호에 따라 CFAR parameter의 최적화가 필요
    - OS-CFAR Block Diagram (Order-static CFAR)
        - Reference cell들을 오름차순 정렬하여, n번째 cell 값(약 3/4번째 값) * threshold 과 test cell 값을 비교 -> Test cell 값이 크면 Target으로 탐지
        - 주위의 잡음 신호에 따라 threshold를 유동적으로 설정
- 2D OS-CFAR in Range-Doppler Heatmap
    - 2D OS-CFAR
        - Range와 Doppler(or Azimuth) 차원 간의 상관 관계를 고려
        - **몇 번째 Doppler bin과 Range bin에 타겟이 위치해 있는지 탐지할 수 있음**
    - CFAR Parameter list in Range-Doppler Heatmap
        - reference_row, reference_col = reference cell 개수 (row, col)
        - guard_row, guard_col = guard cell 개수 (row, col)
        - alpha_os = threshold scale factor (detection factor)
        - **Parameter 최적화는 위젯으로 돌려보면서! + Frame 여러 개에서 잘 탐지해야 함**

```
if radar_no == 'radar_0':
        reference_row = 7
        guard_row = 5 #4
        reference_col = 5
        guard_col = 5
        alpha_os = 2

elif radar_no == 'radar_1':
        reference_row = 7
        guard_row = 4
        reference_col = 5
        guard_col = 5
        alpha_os = 2.1
```

- - **Extended** Range-Doppler Heatmap에서 2D CFAR 적용 -> **Target이 있는 Chirp index 검출**
        - Dynamic Scene의 경우 **Target이 있는 Chirp index에서만 Bartlett 처리**
            - Range Point와 Doppler Point로 나눠서 처리
            - Extended Heatmap으로 Detection -> 유효한 Detection 결과로 재구성
                - Valid Index = chirp/2 ~ chirp+chirp/2 (32번째 ~ 95번째 index) 데이터만 사용

```
T_os, peak_idx_os = OS_CFAR_2D((doppler_heatmap_extended[frameIDX, :, 0, :]),
                                                        num_ref_row=reference_row,
num_guard_row=guard_row,
                                                        num_ref_col=reference_col,
num_guard_col=guard_col,
                                                        set_alpha_os=alpha_os)

detected_range_point = [x[0] for x in peak_idx_os]
detected_doppler_point = [x[1] for x in peak_idx_os]

valid_indices = []
for i in range(len(detected_range_point)):
        if int(numChirp/2) <= detected_doppler_point[i] <= (numChirp+int(numChirp/2))-1:
```

```
                valid_indices.append(i)

CFAR_range_point_moving_obj = [detected_range_point[i] for i in valid_indices]
CFAR_doppler_point_moving_obj = [detected_doppler_point[i] for i in valid_indices]
# numChirp = 64보다 작으면 그대로 idx 넣고 numChirp보다 크면 idx-numChirp 해서 넣음
CFAR_doppler_point_moving_obj = [idx if idx <= numChirp - 1 else idx - numChirp for idx
                                                                    in
CFAR_doppler_point_moving_obj]
```

9. Applying Bartlett Processing

- in Dynamic Scene
  - **CFAR 결과로 나온 Range, Doppler point 있는 데이터만** Bartlett 연산 수행
    - Range FFT -> MTI Filtering -> Doppler FFT 처리한 데이터에서 처리
      - MTI_RAI = MTI Range-Angle Image *(Detected Range, 181(Angle))*
      - DoA = Range 하나에 대한 DoA 결과

```
MTI_RAI = None
DoA = None
incident_angles = None

if CFAR_range_point_moving_obj and CFAR_doppler_point_moving_obj:
        MTI_RAI, DoA, incident_angles = apply_bartlett_moving_target
        (dynamic_scene_range_FFT_data, CFAR_range_point_moving_obj,
        CFAR_doppler_point_moving_obj, frameIDX)
```

- in Static Scene
  - Frame=frameIDX, Chirp=chirpIDX (32, **Doppler=0인 index**)에서 Bartlett 연산 수행
    - Range FFT -> Doppler FFT 처리한 데이터에서 처리
      - direct_RAI = static Range-Angle Image *(Range, 181(Angle))*
      - DoA_static = Range 하나에 대한 DoA 결과

```
direct_RAI, DoA_static, incident_angles_static =
apply_bartlett_static(static_scene_range_FFT_data, chirpIDX, frameIDX)
```

- CFAR Detection을 위한 Extended Heatmap
  - Extended RAI -> Angle 영역이 두 배로 확장됨

```
try:
        MTI_RAI_extended = np.concatenate((MTI_RAI, MTI_RAI), axis= -1)

except:
        pass
direct_RAI_extended = np.concatenate((direct_RAI, direct_RAI), axis= -1)
```

- Angle Setting
  - angle이 angle_set보다 작으면 그대로 idx 넣고 angle_set보다 크면 idx-angle_set 해서 넣음
  - Extended RAI로 Detection하기 때문에 유효하게 0~180의 각도로 재구성

```
angle_set = 181 # for RAIs
```

10. 1D CFAR Processing (in Dynamic Scene)

- 1D CFAR Processing
  - Range마다 Angle 방향으로 CFAR Detection -> 어느 Angle에 target이 있는지 알 수 있음
    - Range bin마다 연관성 X (Target 있는 Range point만 뽑아서 CFAR 돌리기 때문)
    - Target이 있는 Range, Doppler, Angle 모두 알 수 있음
  - CFAR Parameter list (1D)
    - reference_1d = Reference cell 개수
    - guard_1d = Guard cell 개수
    - alpha_os_1d = Threshold factor

```
if radar_no == 'radar_0':
        reference_1D = 30
        guard_1D = 5

        alpha_os_1D = 12.5

elif radar_no == 'radar_1':
        reference_1D = 33
        guard_1D = 5

        alpha_os_1D = 3.5
```

-
  - 1D CFAR Processing -> Moving Target Detection
    - Extended MTI RAI를 사용해서 CFAR Detection

```
detected_range_point_dynamic = []
detected_angle_point_dynamic = []

for i in range(len(CFAR_range_point_moving_obj)):
        T_os_1D, peak_idx_os_1D = OS_CFAR_1D(np.abs(MTI_RAI_extended[i, :]),
        num_ref = reference_1D, num_guard = guard_1D, set_alpha_os = alpha_os_1D)

        len_peak_range_os = len(peak_idx_os_1D)

        if len_peak_range_os == 0:
                detected_range_point_dynamic.append(None)
                detected_angle_point_dynamic.append(None)

        else:
                for j in range(len_peak_range_os):
                        detected_range_point_dynamic.append(i)
                        detected_angle_point_dynamic.append(peak_idx_os_1D[j])

valid_MTI_RAI = []
for i in range(len(detected_range_point_dynamic)):
        if detected_angle_point_dynamic[i] is not None and int(angle_set/2) <=
        detected_angle_point_dynamic[i] <= (angle_set+int(angle_set/2))-1:
                valid_MTI_RAI.append(i)
```

```
CFAR_range_point_1D = [detected_range_point_dynamic[i] for i in valid_MTI_RAI]
CFAR_angle_point_1D = [detected_angle_point_dynamic[i] for i in valid_MTI_RAI]
CFAR_angle_point_1D = [idx if idx <= angle_set - 1 else idx - angle_set for idx in
                                                CFAR_angle_point_1D]
```

11. 2D CFAR Processing (in Static Scene)

- 2D CFAR Processing
  - Range-Angle 방향으로 2D CFAR Detection
    - Range bin마다 연관성 O
    - Target이 있는 Range, Doppler, Angle 모두 알 수 있음
  - CFAR Parameter list (2D)

```
if radar_no == 'radar_0':
        reference_row_static = 18
        guard_row_static = 5
        reference_col_static = 30
        guard_col_static = 7
        alpha_os_static = 8

elif radar_no == 'radar_1':
        reference_row_static = 9
        guard_row_static = 5
        reference_col_static = 11
        guard_col_static = 7
        alpha_os_static = 10
```

-
  - 2D CFAR Processing -> Static Target Detection
    - Extended direct RAI를 사용하여 CFAR Detection

```
T_os_static, peak_idx_os_static = OS_CFAR_2D(np.abs(direct_RAI_extended[:, :]),

num_ref_row=reference_row_static,

num_guard_row=guard_row_static,

num_ref_col=reference_col_static,

num_guard_col=guard_col_static,

set_alpha_os=alpha_os_static)

detected_range_point_static = [x[0] for x in peak_idx_os_static]
detected_angle_point_static = [x[1] for x in peak_idx_os_static]

valid_direct_RAI = []
for i in range(len(detected_range_point_static)):
        if int(angle_set/2) <= detected_angle_point_static[i] <=
        (angle_set+int(angle_set/2))-1:
                valid_direct_RAI.append(i)
```

```
CFAR_range_point_static = [detected_range_point_static[i] for i in valid_direct_RAI]
CFAR_angle_point_static = [detected_angle_point_static[i] for i in valid_direct_RAI]
CFAR_angle_point_static = [idx if idx <= angle_set - 1 else idx - angle_set for idx in
CFAR_angle_point_static]
```

12. Point Combination

Info

- Point Cloud의 Point 별 목표물 정보
  - Timestamp: 기록된 시간
  - Range: 레이다로부터의 거리 [m]
  - Azimuth: 레이다로부터의 방위각 [deg]
  - RCS: 물체의 반사도 [dB]
  - x, y, z: 차량 기준 좌표(Global Coordinate) [m]
  - v_r: 레이다 방사방향 속도
  - v_x, v_y: 차량 이동을 보상한 x방향과 y방향의 속도

- Velocity Axis
  - Doppler axis -> Velocity axis
  - -Max Velocity ~ Max Velocity를 numChirp개로 나눠서 Velocity bin 생성 (linspace)

```
velocity_axis = np.linspace(-Max_Velocity, Max_Velocity, numChirp)
```

- Data Zipping
  - zip() 함수로 두 개의 데이터를 짝 지어줌
  - in Dynamic Scene: (Range, Doppler) / (Range, Angle) Data
    - (Range, Doppler): 2D CFAR 결과로 얻어낸 Target의 (Range, Doppler) index
      - -> Doppler data를 Velocity data로 바꿔서 (Range, Velocity) data로 만듦
    - (Range, Angle): 1D CFAR 결과로 얻어낸 Target의 (Range, Angle) index
  - in Static Scene: (Range, Angle) Data
    - (Range, Angle): 2D CFAR 결과로 얻어낸 Target의 (Range, Angle) index

```
## Dynamic Scene Data (Range, Doppler) -> (Range, Velocity)
dynamic_range_data = CFAR_range_point_moving_obj # 2D CFAR 결과 Range data
doppler_data = CFAR_doppler_point_moving_obj
velocity_data = velocity_axis[CFAR_doppler_point_moving_obj]

dynamic_range_velocity_data = list(zip(dynamic_range_data, velocity_data))

## Dynamic Scene Data (Range, Angle)
dynamic_CFAR_range_data = CFAR_range_point_1D
dynamic_angle_data = CFAR_angle_point_1D
cfar_range_angle_data = list(zip(dynamic_CFAR_range_data, dynamic_angle_data))

## Static Scene Data (Range, Angle)
```

```
static_range_data = CFAR_range_point_static

static_CFAR_range_data = CFAR_range_point_static
static_angle_data = CFAR_angle_point_static
static_cfar_range_angle_data = list(zip(static_CFAR_range_data, static_angle_data))
```

- Data Combining [in Dynamic Scene]
  - 검출된 Moving Target의 Range, Angle, z, RCS, Velocity를 가지는 dynamic_total_data

```
dynamic_total_data_range = []
dynamic_total_data_velocity = []
dynamic_total_data_RCS = []

for cfar_range, angle in zip(dynamic_CFAR_range_data, dynamic_angle_data):
        if cfar_range is not None:
                range_idx = dynamic_range_data[cfar_range]
                velocity = velocity_data[cfar_range]
                RCS_dynamic = moving_obj_RCS = 10*np.log(np.abs(MTI_RAI[cfar_range, angle]))

                dynamic_total_data_range.append(range_idx)
                dynamic_total_data_velocity.append(velocity)
                dynamic_total_data_RCS.append(RCS_dynamic)

        else:
                dynamic_total_data_range.append(None)
                dynamic_total_data_velocity.append(None)
                dynamic_total_data_RCS.append(None)

# r != None 이면 0, r = None이면 None
z_dynamic = [0 if r is not None else None for r in dynamic_total_data_range]
dynamic_total_data = list(zip(dynamic_total_data_range, dynamic_angle_data, z_dynamic,
                                                        dynamic_total_data_RCS,
dynamic_total_data_velocity))
                                                        # range, angle, z, RCS, v
```

- Data Combining [in Static Scene]
  - 검출된 Static Target의 Range, Angle, z, RCS, Velocity를 가지는 static_total_data

```
static_total_data_range = []
static_total_data_RCS = []

for cfar_range_static, angle_static in zip(static_CFAR_range_data, static_angle_data):
        if cfar_range_static is not None:
                range_idx_static = cfar_range_static
                RCS_static = stationary_object_RCS =
                10*np.log(np.abs(direct_RAI[cfar_range_static, angle_static]))

                static_total_data_range.append(range_idx_static)
                static_total_data_RCS.append(RCS_static)

        else:
                static_total_data_range.append(None)
                static_total_data_RCS.append(None)
```

```
static_total_data_velocity = [0 if r is not None else None for r in
                                                                static_total_data_range]
z_static = [0 if r is not None else None for r in static_total_data_range]

static_total_data = list(zip(static_total_data_range, static_angle_data, z_static,
                                                              static_total_data_RCS,
static_total_data_velocity))
                                                              # range, angle, z, RCS, v
```

- Coordinate Point Cloud
  - Point Cloud의 위치 Conversion
  - Range, Angle (Radial) -> x, y global coordinate
    - range = range_meters = r * range_resolution

```
def coordinates_point_cloud(x, y, range_resolution, num_range_bins):
        max_range = range_resolution * num_range_bins # Range bin 1칸 = Range resolution
        x_values = []
        y_values = []

        for theta, r in zip(x, y):
                if r is None or theta is None:
                        x_values.append(None)
                        y_values.append(None)
                        continue

                range_meters = r * range_resolution  # r (Range resolution bin) 1개 = 0.07m

                x_coord = range_meters * math.sin(math.radians(theta-90))
                y_coord = range_meters * math.cos(math.radians(theta-90))

                x_values.append(x_coord)
                y_values.append(y_coord)

        return x_values, y_values
```

- Coordinate Heatmap
  - RAI (Range-Angle Image) Heatmap의 Point를 x, y 좌표계로 변경

```
def coordinates_heatmap(data, range_resolution):
        ranges = np.arange(data.shape[0]) * range_resolution
        angles = np.linspace(-np.pi/2, np.pi/2, data.shape[1], endpoint=False)

        x = np.outer(ranges, np.cos(angles))
        y = np.outer(ranges, np.sin(angles))

        return x, y
```

- Extracted Data (Range, Angle) -> Converted Data (x, y)
  - Range, Angle (r, ϕ) -> (x, y)로 Coordinate Conversion
  - Total Data: (x, y, z, RCS, v) 순서로 Point cloud data가 들어가 있음

- LOS Clutter + Moving Target

```
# index 0 = Range, index 1 = Angle for dynamic_total_data
extracted_dynamic_range = [data[0] for data in dynamic_total_data]
extracted_dynamic_angle = [data[1] for data in dynamic_total_data]

# index 0 = Range, index 1 = Angle for static_total_data
extracted_static_range = [data[0] for data in static_total_data]
extracted_static_angle = [data[1] for data in static_total_data]

# Convert Range, Angle -> x, y coordinate

converted_dynamic_point_x, converted_dynamic_point_y =
coordinates_point_cloud(extracted_dynamic_angle, extracted_dynamic_range,
                                                range_resolution=0.07, num_range_bins=256)

converted_static_point_x, converted_static_point_y =
coordinates_point_cloud(extracted_static_angle, extracted_static_range,
                                                range_resolution=0.07, num_range_bins=256)

converted_static_RAI_y, converted_static_RAI_x =
coordinates_heatmap(direct_RAI,range_resolution=0.07)

# (x, y, z, RCS, v) 순서로 dynamic, static scene의 total data 만들어줌

for i in range(len(dynamic_total_data)):
        dynamic_total_data[i] = (converted_dynamic_point_x[i],
        converted_dynamic_point_y[i]) + dynamic_total_data[i][2:] # need to check

for i in range(len(static_total_data)):
        static_total_data[i] = (converted_static_point_x[i], converted_static_point_y[i]) +
        static_total_data[i][2:]

dynamic_final_data = dynamic_total_data
static_final_data = static_total_data

# Remove None
dynamic_final_data = [data for data in dynamic_final_data if None not in data]
static_final_data = [data for data in static_final_data if None not in data]
```

13. Velocity Compensation (in Moving Vehicle)

- Ego-Motion Distortion
  - Radar 센서가 움직이면 (Moving Ego) 정지된 물체들은 Radar와 반대 방향으로 이동하는 것처럼 보임
    - 정지된 물체들의 상대적인 속도는 Radar의 속도와 Heading 방향이 정확히 같음
  - Radar는 방사형으로 타겟을 탐지 -> 타겟의 Radial한 속도만 측정이 가능
  - Ego-Motion으로 인해 타겟의 속도에 왜곡이 발생 -> Velocity Compensation 필요
  - (in Velocity-Angle Map) 중심 기준으로 끝으로 갈수록 Radial Velocity가 0에 수렴
    - 각도마다 Radial Velocity Calibration을 다르게 해야 함
- Velocity Compensation
  - 제안 방식

- Global Coordinate 기반
  - 1. Target의 Point를 기준으로 Point의 Radial Ego-Velocity를 계산
  - 2. 구한 Radial Ego-Velocity를 x, y 성분으로 나누어 Point Velocity에서 차감
- **Ego-Velocity가 x, y 성분으로 구성 -> 각도에 따른 Radial Velocity 반영 가능**
- 1. Radial Ego-Velocity
  - Point를 기준으로 Radial Ego-Velocity를 구함
- 2. Radial Ego-Velocity Decomposition
  - x, y 성분으로 Decomposition
  - Point 각도 p_ϕ에 따른 Radial Velocity 반영 가능
- 3. Point Velocity Subtraction
  - 측정한 Point Velocity에서 센서의 Radial Velocity를 빼서 Calibrate한 Point의 Velocity를 구함

- Velocity Compensation Function
  - Input: Global Coordinate Point Data (x, y coordinate)
  - 결과: 속도 보정 오차가 일부 줄어듦 확인

```
def vel_compensation(x_y_input):
        '''
        Function for compensate velocity of points
        Use this function as default!!

        x_y_input : global coordinate(x, y)
        '''

        x_y_azi=np.zeros((x_y_input.shape[0],1),dtype=float)
        ego_vel_point=np.zeros((x_y_input.shape[0],1),dtype=float) # test

        df_vel=df_point["v"]
        cal_vel=np.zeros((x_y_input.shape[0],1), dtype=float)
        target_pos=np.zeros((x_y_input.shape[0],2), dtype=float)
        sensor_vel=np.zeros((x_y_input.shape[0],2), dtype=float)

        for i in range(x_y_input.shape[0]):
                x_y_azi[i]=math.atan(x_y_input[i,1]/x_y_input[i,0])
                if x_y_azi[i]<0:
                        x_y_azi[i]=x_y_azi[i]+np.pi

                ego_vel_point[i] = ego_vel * math.cos((np.pi/2)-x_y_azi[i]) # test

                if x_y_azi[i]<=90*np.pi/180:
                        sensor_vel[i,:]=(ego_vel_point[i][0] *
                        math.cos(x_y_azi[i])+wheelbase*yaw_rate, ego_vel_point[i][0] *
                        math.sin(x_y_azi[i])+(-overall_width/2)*yaw_rate)
                        # default : yaw_rate = 0 --> sensor_vel = ego_vel # test
                else:
                        sensor_vel[i,:]=(-ego_vel_point[i][0] *
                        math.cos(x_y_azi[i])+wheelbase*yaw_rate, ego_vel_point[i][0] *
                        math.sin(x_y_azi[i])+(-overall_width/2)*yaw_rate)
                        # default : yaw_rate = 0 --> sensor_vel = ego_vel # test

                target_pos[i]=((math.cos(x_y_azi[i]+yaw_rate)),
                np.abs(math.sin(x_y_azi[i]+yaw_rate)))
```

```
                cal_vel[i]=df_vel[i]-(-sensor_vel[i])@target_pos[i,:]

                if np.round(np.abs(cal_vel[i]))<=0.5 or df_vel[i]==0: # added for jitter &
ground reflection
                        cal_vel[i]=0

        return cal_vel
```

- Applying Velocity Compensation
  - Point의 Velocity를 다 불러와서 Velocity Calibration

```
for numPoint in matched_frame_numbers:

        numPoint_str = str(numPoint).zfill(6)
        wheel_filepath = os.path.join(point_cloud_check_dir,
        f'{data_file_name}_{numPoint_str}th_frame.csv').replace("\\","/")

        matching_row = df_matched[df_matched['radar_frame'] == numPoint]

        if not matching_row.empty:
                data_value = matching_row.iloc[0, 1]  # 행 번호로 선택: iloc
                # test(wheel = [0, 1], radar = [0, 3], lidar = [0, 5])

                if data_value <= 0: ##
                        data_value = -data_value
                else:
                        data_value = data_value ##

            numPoint_str = str(numPoint).zfill(6)
            wheel_filepath = os.path.join(point_cloud_check_dir,
            f'{data_file_name}_{numPoint_str}th_frame.csv').replace("\\", "/")

            df_point = pd.read_csv(wheel_filepath)

            point_cloud_save_dir_csv = f'./{data_folder_name}/{radar_no}/csv'

            if not os.path.exists(point_cloud_save_dir_csv):
                    os.makedirs(point_cloud_save_dir_csv)
                try:
                        ego_vel = data_value
                except:
                        ego_vel = 0

                point_cc = global_coordinate(df_point, overall_width = overall_width,
                wheelbase = wheelbase, theta_rad = theta_rad, yaw_rate = yaw_rate)
                # local to global coordinate

                x_y_direction = (point_cc[:2, :]).T

                cal_vel = vel_compensation(x_y_direction) # velocity compensation
                # cal_vel = vel_compensation_local(df_point)
```